

PRACTICAL-04

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:-

```
srikar@LAPTOP-KHLR30TF +  
srikar@LAPTOP-KHLR30TF:~/practical$ cat practical-04.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t child1, child2, child3;  
    int status;  
  
    printf("Parent Process\n");  
    printf("Parent PID: %d\n", getpid());  
  
    // Create Child 1  
    child1 = fork();  
  
    if (child1 == 0)  
    {  
        printf("Child 1: PID = %d, PPID = %d\n", getpid(), getppid());  
        sleep(3);  
        printf("Child 1 completed\n");  
        exit(10);  
    }  
  
    // Create Child 2  
    child2 = fork();  
  
    if (child2 == 0)  
    {  
        printf("Child 2: PID = %d, PPID = %d\n", getpid(), getppid());  
        sleep(5);  
        printf("Child 2 completed\n");  
        exit(20);  
    }  
  
    // Create Child 3  
    child3 = fork();  
  
    if (child3 == 0)  
    {  
        child3 = fork();  
  
        if (child3 == 0)  
        {  
            printf("Child 3: PID = %d, PPID = %d\n", getpid(), getppid());  
            sleep(2);  
            printf("Child 3 completed\n");  
            exit(30);  
        }  
  
        // Parent waits for any child  
        printf("\nParent using wait()\n");  
        pid_t completed = wait(&status);  
  
        if (WIFEXITED(status))  
        {  
            printf("wait(): Child PID %d completed with status %d\n",  
                completed, WEXITSTATUS(status));  
        }  
  
        // Parent waits specifically for Child 2  
        printf("\nParent using waitpid() for Child 2\n");  
        waitpid(child2, &status, 0);  
  
        if (WIFEXITED(status))  
        {  
            printf("waitpid(): Child 2 PID %d completed with status %d\n",  
                child2, WEXITSTATUS(status));  
        }  
  
        // Wait for remaining children  
        wait(NULL);  
  
        printf("\nAll child processes completed.\n");  
        printf("Parent process terminating.\n");  
  
        return 0;  
    }  
}
```

OUTPUT:-

```
srikar@LAPTOP-KHLR30TF:~/lab/practical$ nano practical-04.c
srikar@LAPTOP-KHLR30TF:~/lab/practical$ gcc practical-04.c -o practical
srikar@LAPTOP-KHLR30TF:~/lab/practical$ ./practical-04
-bash: ./practical-04: No such file or directory
srikar@LAPTOP-KHLR30TF:~/lab/practical$ ./practical-04
-bash: ./practical-04: No such file or directory
srikar@LAPTOP-KHLR30TF:~/lab/practical$ gcc practical-04.c -o practical-04
srikar@LAPTOP-KHLR30TF:~/lab/practical$ ./practical-04
Parent Process
Parent PID: 956
Child 1: PID = 957, PPID = 956
Child 2: PID = 958, PPID = 956
Parent using wait()
Child 3: PID = 959, PPID = 956
Child 3 completed
wait(): Child PID 959 completed with status 30
Parent using waitpid() for Child 2
Child 1 completed
Child 2 completed
waitpid(): Child 2 PID 958 completed with status 20
All child processes completed.
Parent process terminating.
srikar@LAPTOP-KHLR30TF:~/lab/practical$ cat practical-04.c
```